UNIVERSITÉ SORBONNE PARIS NORD

INSTITUT GALILÉE

Base de données avancées

TP1- Quelques fonctions SQL et requêtes

Fait par :

Khaoula Charef

Numéro Etudiant :

12509041

Formation

Ingénierie Informatique

Exercice 1 :

Question 1 : Définition de la table `section`

La table `section` a été définie dans le script de création avec les attributs demandés. L'attribut `semester` possède une contrainte de domaine limitant ses valeurs à `Fall`, `Winter`, `Spring` et `Summer`. La clé primaire est composée de `course_id`, `sec_id`, `semester` et `year`. Des clés étrangères assurent également la cohérence avec les tables `course` et `classroom`.

```
SQL> CREATE TABLE section(
  2      course_id    VARCHAR(8),
  3      sec_id       VARCHAR(8),
  4      semester     VARCHAR(6)
  5          CHECK (semester IN ('Fall', 'Winter', 'Spring', 'Summer')),
  6      year         NUMERIC(4,0),
  7      building     VARCHAR(15),
  8      room_number  VARCHAR(7),
  9      time_slot_id VARCHAR(4),
 10      PRIMARY KEY (course_id, sec_id, semester, year),
 11      FOREIGN KEY (course_id) REFERENCES course,
 12      FOREIGN KEY (building, room_number) REFERENCES classroom
 13  );
```

FIGURE 1 – Définition SQL de la table `section`

Question 2 : Diagramme entité-association

À partir du schéma relationnel fourni dans le sujet, on identifie les principales entités suivantes : `department`, `course`, `teacher`, `student`, `classroom`, `section` et `time_slot`.

Les relations principales entre ces entités sont :

- un département propose plusieurs cours ;
- un département regroupe plusieurs enseignants et plusieurs étudiants ;
- un cours peut être ouvert sous forme de plusieurs sections ;
- une section se déroule dans une salle et à un créneau horaire donné ;
- un enseignant peut assurer plusieurs sections via la relation `teaches` ;
- un étudiant peut suivre plusieurs sections via la relation `takes` ;
- un étudiant peut avoir un enseignant référent via la relation `advisor` ;
- un cours peut avoir un ou plusieurs prérequis via la relation `prereq`.

Le schéma met en évidence que la base modélise une gestion universitaire centrée sur les cours, leurs ouvertures, les enseignants et les inscriptions des étudiants.

Question 3 : Création et peuplement de la base

La base a été créée à l'aide du script `createschema.sql`, puis peuplée avec le script `data.sql`. L'exécution de ces scripts sous `sqlplus` se fait avec la commande `@nom_script`. Une vérification a ensuite été réalisée en affichant la liste des tables et le contenu de certaines relations, comme `course`.

```
SQL> SELECT table_name FROM user_tables;

TABLE_NAME
-----------------------------------------------------------
CLASSROOM
DEPARTMENT
COURSE
TEACHER
SECTION
TEACHES
STUDENT
TAKES
ADVISOR
TIME_SLOT
PREREQ

11 lignes sÚlectionnÚes.
```

FIGURE 2 – Création du schéma sous SQL*Plus

```
SQL> select * from course;

COURSE_I|TITLE                                          |DEPT_NAME           |   CREDITS
--------|-----------------------------------------------|--------------------|----------
BIO-101 |Intro. to Biology                              |Biology             |         4
BIO-301 |Genetics                                       |Biology             |         4
BIO-399 |Computational Biology                          |Biology             |         3
CS-101  |Intro. to Computer Science                     |Comp. Sci.          |         4
CS-190  |Game Design                                    |Comp. Sci.          |         4
CS-315  |Robotics                                       |Comp. Sci.          |         3
CS-319  |Image Processing                               |Comp. Sci.          |         3
CS-347  |Database System Concepts                       |Comp. Sci.          |         3
EE-181  |Intro. to Digital Systems                      |Elec. Eng.          |         3
FIN-201 |Investment Banking                             |Finance             |         3
HIS-351 |World History                                  |History             |         3
MU-199  |Music Video Production                         |Music               |         3
PHY-101 |Physical Principles                            |Physics             |         4

13 lignes sÚlectionnÚes.

SQL>
SQL> select * from department;

DEPT_NAME           |BUILDING       |    BUDGET
--------------------|---------------|----------
Biology             |Watson         |     90000
Comp. Sci.          |Taylor         |    100000
Elec. Eng.          |Taylor         |     85000
Finance             |Painter        |    120000
History             |Painter        |     50000
Music               |Packard        |     80000
Physics             |Watson         |     70000
```

FIGURE 3 – Insertion des données sous SQL*Plus

Question 4 : Insertion du cours `BIO-101`

L'énoncé demande l'insertion du cours `BIO-101`. Avant toute insertion, une vérification a été effectuée dans la table `course`. Cette vérification montre que ce cours est déjà présent dans les données de peuplement. Par conséquent, une nouvelle insertion de la même ligne provoquerait une erreur de duplication de clé primaire.

La requête demandée est la suivante :

```
insert into course values ('BIO-101', 'Intro. to Biology', 'Biology', 4);
```

```
SQL> select * from course where course_id = 'BIO-101';

COURSE_I|TITLE                                          |DEPT_NAME           |   CREDITS
--------|-----------------------------------------------|--------------------|----------
BIO-101 |Intro. to Biology                              |Biology             |         4
```

Figure 4 – Vérification de l'existence du cours `BIO-101`

Exercice 2

Question 1 : Structure et contenu de la relation `section`

La première étape consiste à afficher la structure de la relation `section`, puis son contenu.

```
DESC section;
SELECT * FROM section;
```

La commande `DESC` permet d'obtenir les attributs de la table ainsi que leurs types. La requête `SELECT *` affiche l'ensemble des tuples de la relation `section`, c'est-à-dire les différents cours proposés selon le semestre, l'année, la salle et le créneau horaire.

```
SQL> desc section;
 Nom                                       NULL ?   Type
 ----------------------------------------- -------- ----------------------------
 COURSE_ID                                 NOT NULL VARCHAR2(8)
 SEC_ID                                    NOT NULL VARCHAR2(8)
 SEMESTER                                  NOT NULL VARCHAR2(6)
 YEAR                                      NOT NULL NUMBER(4)
 BUILDING                                           VARCHAR2(15)
 ROOM_NUMBER                                        VARCHAR2(7)
 TIME_SLOT_ID                                       VARCHAR2(4)
```

Figure 5 – Structure de la table `section`

```
SQL> SELECT * from  section;

COURSE_I|SEC_ID  |SEMEST|      YEAR|BUILDING       |ROOM_NU|TIME
--------|--------|------|----------|---------------|-------|----
BIO-101 |1       |Summer|      2009|Painter        |514    |B
BIO-301 |1       |Summer|      2010|Painter        |514    |A
CS-101  |1       |Fall  |      2009|Packard        |101    |H
CS-101  |1       |Spring|      2010|Packard        |101    |F
CS-190  |1       |Spring|      2009|Taylor         |3128   |E
CS-190  |2       |Spring|      2009|Taylor         |3128   |A
CS-315  |1       |Spring|      2010|Watson         |120    |D
CS-319  |1       |Spring|      2010|Watson         |100    |B
CS-319  |2       |Spring|      2010|Taylor         |3128   |C
CS-347  |1       |Fall  |      2009|Taylor         |3128   |A
EE-181  |1       |Spring|      2009|Taylor         |3128   |C
FIN-201 |1       |Spring|      2010|Packard        |101    |B
HIS-351 |1       |Spring|      2010|Painter        |514    |C
MU-199  |1       |Spring|      2010|Packard        |101    |D
PHY-101 |1       |Fall  |      2009|Watson         |100    |A

15 lignes sÚlectionnÚes.
```

FIGURE 6 – Contenu de la table `section`

Question 2 : Affichage de la relation `course`

La requête suivante affiche tous les renseignements sur les cours que l'on peut programmer.

```
SELECT * FROM course;
```

Elle permet d'obtenir l'identifiant du cours, son titre, le département responsable et le nombre de crédits associés.

```
SQL> SELECT * FROM course;

COURSE_I|TITLE                                          |DEPT_NAME          |   CREDITS
--------|-----------------------------------------------|-------------------|----------
BIO-101 |Intro. to Biology                              |Biology            |         4
BIO-301 |Genetics                                       |Biology            |         4
BIO-399 |Computational Biology                          |Biology            |         3
CS-101  |Intro. to Computer Science                     |Comp. Sci.         |         4
CS-190  |Game Design                                    |Comp. Sci.         |         4
CS-315  |Robotics                                       |Comp. Sci.         |         3
CS-319  |Image Processing                               |Comp. Sci.         |         3
CS-347  |Database System Concepts                       |Comp. Sci.         |         3
EE-181  |Intro. to Digital Systems                      |Elec. Eng.         |         3
FIN-201 |Investment Banking                             |Finance            |         3
HIS-351 |World History                                  |History            |         3
MU-199  |Music Video Production                         |Music              |         3
PHY-101 |Physical Principles                            |Physics            |         4

13 lignes sÚlectionnÚes.
```

FIGURE 7 – Contenu de la relation `course`

Question 3 : Titres des cours et départements

La requête suivante permet d'afficher les titres des cours ainsi que les départements qui les proposent.

```
SELECT title, dept_name FROM course;
```

```
SQL> SELECT title, dept_name from course;

TITLE                                             |DEPT_NAME
--------------------------------------------------|-----------------
Intro. to Biology                                 |Biology
Genetics                                          |Biology
Computational Biology                             |Biology
Intro. to Computer Science                        |Comp. Sci.
Game Design                                       |Comp. Sci.
Robotics                                          |Comp. Sci.
Image Processing                                  |Comp. Sci.
Database System Concepts                          |Comp. Sci.
Intro. to Digital Systems                         |Elec. Eng.
Investment Banking                                |Finance
World History                                     |History
Music Video Production                            |Music
Physical Principles                               |Physics

13 lignes sÚlectionnÚes.
```

FIGURE 8 – Titres des cours et départements correspondants

Question 4 : Départements et budgets

La requête suivante affiche les noms des départements ainsi que leur budget.

```
SELECT dept_name, budget FROM department;
```

```
SQL> SELECT dept_name, budget from department;

DEPT_NAME           |    BUDGET
--------------------|----------
Biology             |     90000
Comp. Sci.          |    100000
Elec. Eng.          |     85000
Finance             |    120000
History             |     50000
Music               |     80000
Physics             |     70000

7 lignes sÚlectionnÚes.
```

FIGURE 9 – Noms des départements et leurs budgets

Question 5 : Enseignants et départements

La requête suivante permet d'afficher le nom de chaque enseignant ainsi que son département.

```
SELECT name, dept_name FROM teacher;
```

```
SQL> SELECT name, dept_name from teacher;

NAME                 |DEPT_NAME
---------------------|--------------------
Srinivasan           |Comp. Sci.
Wu                   |Finance
Mozart               |Music
Einstein             |Physics
El Said              |History
Gold                 |Physics
Katz                 |Comp. Sci.
Califieri            |History
Singh                |Finance
Crick                |Biology
Brandt               |Comp. Sci.
Kim                  |Elec. Eng.

12 lignes sÚlectionnÚes.
```

FIGURE 10 – Noms des enseignants et leurs départements

Question 6 : Enseignants ayant un salaire supérieur à 65000

La requête suivante sélectionne les enseignants dont le salaire est strictement supérieur à 65000.

```
SELECT name FROM teacher WHERE salary > 65000;
```

```
SQL> SELECT name FROM teacher WHERE salary > 65000;

NAME
--------------------
Wu
Einstein
Gold
Katz
Singh
Crick
Brandt
Kim

8 lignes sÚlectionnÚes.
```

FIGURE 11 – Enseignants ayant un salaire supérieur à 65000

Question 7 : Enseignants ayant un salaire entre 55000 et 85000

La requête suivante affiche les enseignants dont le salaire est compris entre 55000 et 85000.

```
SELECT name FROM teacher WHERE salary BETWEEN 55000 AND 85000;
```

```
SQL> SELECT name FROM teacher WHERE salary BETWEEN 55000 AND 85000;

NAME
--------------------
Srinivasan
El Said
Katz
Califieri
Singh
Crick
Kim

7 lignes sÚlectionnÚes.
```

FIGURE 12 – Enseignants ayant un salaire compris entre 55000 et 85000

Question 8 : Départements distincts à partir de `teacher`

La requête suivante permet d'afficher les noms des départements présents dans la relation `teacher` sans répétition.

```
SELECT DISTINCT dept_name
FROM teacher;
```

```
SQL> SELECT DISTINCT dept_name FROM teacher;

DEPT_NAME
--------------------
Comp. Sci.
Finance
Music
Physics
History
Biology
Elec. Eng.

7 lignes sÚlectionnÚes.
```

FIGURE 13 – Départements distincts obtenus à partir de la table `teacher`

Question 9 : Enseignants du département informatique ayant un salaire supérieur à 65000

La requête suivante sélectionne les enseignants appartenant au département informatique et dont le salaire dépasse 65000.

```
SELECT name
FROM teacher
WHERE dept_name = 'Comp. Sci.'
  AND salary > 65000;
```

```
SQL> SELECT name FROM teacher WHERE dept_name = 'Comp. Sci.'  AND salary > 65000;

NAME
--------------------
Katz
Brandt
```

FIGURE 14 – Enseignants du département informatique ayant un salaire supérieur à 65000

Question 10 : Cours proposés au printemps 2010

La requête suivante affiche toutes les sections proposées pendant le semestre Spring de l'année 2010.

```
SELECT *
FROM section
WHERE semester = 'Spring'
  AND year = 2010;
```

```
SQL> SELECT * FROM section WHERE semester = 'Spring'  AND year = 2010;

COURSE_I|SEC_ID  |SEMEST|      YEAR|BUILDING       |ROOM_NU|TIME
--------|--------|------|----------|---------------|-------|----
CS-101  |1       |Spring|      2010|Packard        |101    |F
CS-315  |1       |Spring|      2010|Watson         |120    |D
CS-319  |1       |Spring|      2010|Watson         |100    |B
CS-319  |2       |Spring|      2010|Taylor         |3128   |C
FIN-201 |1       |Spring|      2010|Packard        |101    |B
HIS-351 |1       |Spring|      2010|Painter        |514    |C
MU-199  |1       |Spring|      2010|Packard        |101    |D

7 lignes sÚlectionnÚes.
```

FIGURE 15 – Sections proposées au printemps 2010

Question 11 : Titres des cours du département informatique ayant plus de 3 crédits

La requête suivante permet d'afficher les titres des cours proposés par le département informatique et ayant un nombre de crédits strictement supérieur à 3.

```
SELECT title
FROM course
WHERE dept_name = 'Comp. Sci.'
  AND credits > 3;
```

```
SQL> SELECT title FROM course WHERE dept_name = 'Comp. Sci.'  AND credits > 3;

TITLE
--------------------------------------------------
Intro. to Computer Science
Game Design
```

FIGURE 16 – Cours du département informatique ayant plus de 3 crédits

Question 12 : Enseignants, départements et bâtiments

La requête suivante relie les tables `teacher` et `department` afin d'afficher le nom de l'enseignant, son département et le bâtiment où ce département est localisé.

```
SELECT t.name, t.dept_name, d.building
FROM teacher t
JOIN department d
  ON t.dept_name = d.dept_name;
```

```
SQL> SELECT t.name AS teacher_name, t.dept_name AS teacher_depart, d.building AS teacher_building
  2  FROM teacher t
  3  JOIN department d
  4  ON t.dept_name = d.dept_name;

TEACHER_NAME        |TEACHER_DEPART      |TEACHER_BUILDIN
--------------------|--------------------|--------------
Srinivasan          |Comp. Sci.          |Taylor
Wu                  |Finance             |Painter
Mozart              |Music               |Packard
Einstein            |Physics             |Watson
El Said             |History             |Painter
Gold                |Physics             |Watson
Katz                |Comp. Sci.          |Taylor
Califieri           |History             |Painter
Singh               |Finance             |Painter
Crick               |Biology             |Watson
Brandt              |Comp. Sci.          |Taylor
Kim                 |Elec. Eng.          |Taylor

12 lignes sÚlectionnÚes.
```

FIGURE 17 – Enseignants, départements et bâtiments

Question 13 : Étudiants ayant suivi au moins un cours en informatique

La requête suivante permet d'identifier les étudiants ayant suivi au moins un cours rattaché au département informatique.

```
SELECT DISTINCT s.*
FROM student s
JOIN takes tk
  ON s.ID = tk.ID
JOIN course c
  ON tk.course_id = c.course_id
WHERE c.dept_name = 'Comp. Sci.';
```

```
SQL> SELECT DISTINCT s.*
  2  FROM student s
  3  JOIN takes tk
  4  ON s.ID = tk.ID
  5  JOIN course c
  6  ON tk.course_id = c.course_id
  7  WHERE c.dept_name = 'Comp. Sci.';

ID   |NAME                |DEPT_NAME           |  TOT_CRED
-----|--------------------|--------------------|----------
00128|Zhang               |Comp. Sci.          |       102
12345|Shankar             |Comp. Sci.          |        32
45678|Levy                |Physics             |        46
54321|Williams            |Comp. Sci.          |        54
76543|Brown               |Comp. Sci.          |        58
98765|Bourikas            |Elec. Eng.          |        98

6 lignes sÚlectionnÚes.
```

FIGURE 18 – Étudiants ayant suivi au moins un cours en informatique

Question 14 : Étudiants ayant suivi un cours assuré par Einstein

La requête suivante relie les inscriptions des étudiants aux enseignements assurés, puis filtre les sections dispensées par l'enseignant nommé Einstein.

```
SELECT DISTINCT s.name
FROM student s
JOIN takes tk
  ON s.ID = tk.ID
JOIN teaches te
  ON tk.course_id = te.course_id
 AND tk.sec_id = te.sec_id
 AND tk.semester = te.semester
 AND tk.year = te.year
JOIN teacher t
  ON te.ID = t.ID
WHERE t.name = 'Einstein';
```

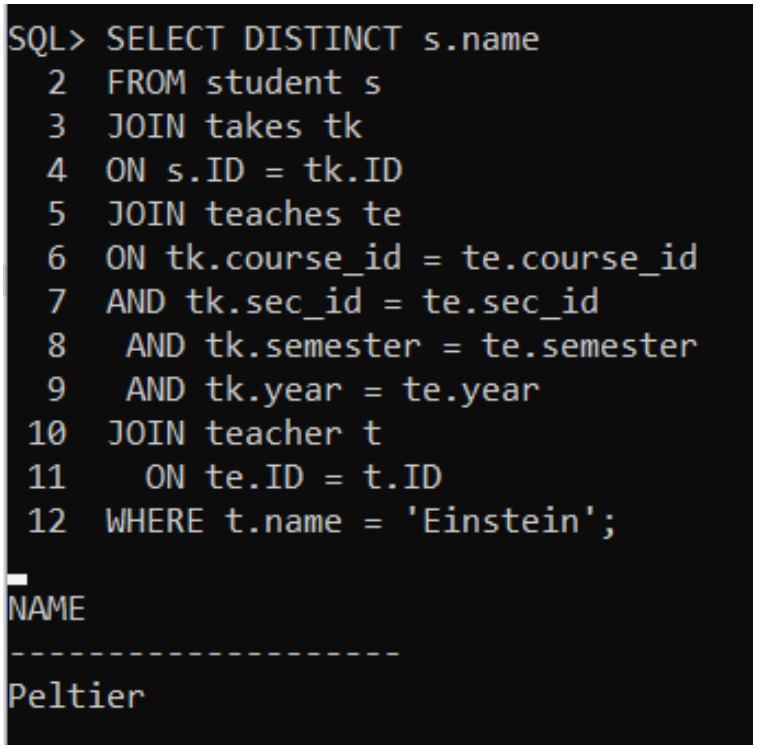

Figure 19 – Étudiants ayant suivi un cours dispensé par Einstein

Question 15 : Cours et enseignants

La requête suivante permet d'associer chaque cours enseigné à l'enseignant qui l'a assuré.

```
SELECT te.course_id, t.name
FROM teaches te
JOIN teacher t
  ON te.ID = t.ID;
```

```
SQL> SELECT te.course_id, t.name
  2  FROM teaches te
  3  JOIN teacher t
  4  ON te.ID = t.ID;

COURSE_I|NAME
--------|--------------------
CS-101  |Srinivasan
CS-315  |Srinivasan
CS-347  |Srinivasan
FIN-201 |Wu
MU-199  |Mozart
PHY-101 |Einstein
HIS-351 |El Said
CS-101  |Katz
CS-319  |Katz
BIO-101 |Crick
BIO-301 |Crick
CS-190  |Brandt
CS-190  |Brandt
CS-319  |Brandt
EE-181  |Kim

15 lignes sÚlectionnÚes.
```

FIGURE 20 – Identifiants des cours et enseignants correspondants

Question 16 : Nombre d'inscrits par enseignement au printemps 2010

La requête suivante calcule le nombre d'étudiants inscrits pour chaque section proposée au semestre Spring 2010.

```
SELECT s.course_id, s.sec_id, s.semester, s.year, COUNT(tk.ID) AS nb_inscrits
FROM section s
LEFT JOIN takes tk
  ON s.course_id = tk.course_id
 AND s.sec_id = tk.sec_id
 AND s.semester = tk.semester
 AND s.year = tk.year
WHERE s.semester = 'Spring'
  AND s.year = 2010
GROUP BY s.course_id, s.sec_id, s.semester, s.year
ORDER BY s.course_id, s.sec_id;
```

```
SQL> SELECT s.course_id, s.sec_id, s.semester, s.year, COUNT(tk.ID) AS nb_inscrits
  2  FROM section s
  3  LEFT JOIN takes tk
  4    ON s.course_id = tk.course_id
  5   AND s.sec_id = tk.sec_id
  6   AND s.semester = tk.semester
  7   AND s.year = tk.year
  8  WHERE s.semester = 'Spring'
  9    AND s.year = 2010
 10  GROUP BY s.course_id, s.sec_id, s.semester, s.year
 11  ORDER BY s.course_id, s.sec_id;

COURSE_I|SEC_ID  |SEMEST|      YEAR|NB_INSCRITS
--------|--------|------|----------|-----------
CS-101  |1       |Spring|      2010|          1
CS-315  |1       |Spring|      2010|          2
CS-319  |1       |Spring|      2010|          1
CS-319  |2       |Spring|      2010|          1
FIN-201 |1       |Spring|      2010|          1
HIS-351 |1       |Spring|      2010|          1
MU-199  |1       |Spring|      2010|          1

7 lignes sÚlectionnÚes.
```

FIGURE 21 – Nombre d'inscrits pour chaque enseignement proposé au printemps 2010

Question 17 : Salaire maximum par département

La requête suivante permet d'afficher, pour chaque département, le salaire maximum parmi les enseignants qui y sont rattachés.

```
SELECT dept_name, MAX(salary) AS salaire_max
FROM teacher
GROUP BY dept_name;
```

```
SQL> SELECT dept_name, MAX(salary) AS salaire_max
  2  FROM teacher
  3  GROUP BY dept_name;

DEPT_NAME           |SALAIRE_MAX
--------------------|-----------
Comp. Sci.          |      92000
Finance             |      90000
Music               |      40000
Physics             |      95000
History             |      62000
Biology             |      72000
Elec. Eng.          |      80000

7 lignes sÚlectionnÚes.
```

FIGURE 22 – Salaire maximum des enseignants par département

Question 18 : Nombre d'inscrits par enseignement proposé

La requête suivante calcule le nombre d'étudiants inscrits dans chaque section proposée.

```
SELECT s.course_id, s.sec_id, s.semester, s.year, COUNT(tk.ID) AS nb_inscrits
FROM section s
LEFT JOIN takes tk
  ON s.course_id = tk.course_id
 AND s.sec_id = tk.sec_id
 AND s.semester = tk.semester
 AND s.year = tk.year
GROUP BY s.course_id, s.sec_id, s.semester, s.year
ORDER BY s.year, s.semester, s.course_id, s.sec_id;
```

```
SQL> SELECT s.course_id, s.sec_id, s.semester, s.year, COUNT(tk.ID) AS nb_inscrits
  2  FROM section s
  3  LEFT JOIN takes tk
  4    ON s.course_id = tk.course_id
  5   AND s.sec_id = tk.sec_id
  6   AND s.semester = tk.semester
  7   AND s.year = tk.year
  8  GROUP BY s.course_id, s.sec_id, s.semester, s.year
  9  ORDER BY s.year, s.semester, s.course_id, s.sec_id;

COURSE_I|SEC_ID  |SEMEST|      YEAR|NB_INSCRITS
--------|--------|------|----------|-----------
CS-101  |1       |Fall  |      2009|          6
CS-347  |1       |Fall  |      2009|          2
PHY-101 |1       |Fall  |      2009|          1
CS-190  |1       |Spring|      2009|          0
CS-190  |2       |Spring|      2009|          2
EE-181  |1       |Spring|      2009|          1
BIO-101 |1       |Summer|      2009|          1
CS-101  |1       |Spring|      2010|          1
CS-315  |1       |Spring|      2010|          2
CS-319  |1       |Spring|      2010|          1
CS-319  |2       |Spring|      2010|          1
FIN-201 |1       |Spring|      2010|          1
HIS-351 |1       |Spring|      2010|          1
MU-199  |1       |Spring|      2010|          1
BIO-301 |1       |Summer|      2010|          1

15 lignes sÚlectionnÚes.
```

FIGURE 23 – Nombre d'inscrits pour chaque enseignement proposé

Question 19 : Nombre total de cours par bâtiment pour Fall 2009 et Spring 2010

La requête suivante compte le nombre total de sections ayant eu lieu dans chaque bâtiment durant les périodes Fall 2009 et Spring 2010.

```
SELECT building, COUNT(*) AS nb_cours
FROM section
WHERE (semester = 'Fall' AND year = 2009)
   OR (semester = 'Spring' AND year = 2010)
GROUP BY building;
```

```
SQL> SELECT building, COUNT(*) AS nb_cours
  2  FROM section
  3  WHERE (semester = 'Fall' AND year = 2009)
  4     OR (semester = 'Spring' AND year = 2010)
  5  GROUP BY building;

BUILDING       |  NB_COURS
---------------|----------
Packard        |         4
Watson         |         3
Taylor         |         2
Painter        |         1
```

FIGURE 24 – Nombre de cours par bâtiment pour Fall 2009 et Spring 2010

Question 20 : Cours dispensés dans le bâtiment du département

La requête suivante calcule, pour chaque département, le nombre total de cours ayant eu lieu dans le bâtiment qui héberge ce département.

```
SELECT d.dept_name, COUNT(*) AS nb_cours
FROM department d
JOIN course c
  ON d.dept_name = c.dept_name
JOIN section s
  ON c.course_id = s.course_id
WHERE s.building = d.building
GROUP BY d.dept_name;
```

```
SQL> SELECT d.dept_name, COUNT(*) AS nb_cours
  2  FROM department d
  3  JOIN course c
  4    ON d.dept_name = c.dept_name
  5  JOIN section s
  6    ON c.course_id = s.course_id
  7  WHERE s.building = d.building
  8  GROUP BY d.dept_name;

DEPT_NAME           |  NB_COURS
--------------------|----------
Comp. Sci.          |         4
Elec. Eng.          |         1
History             |         1
Music               |         1
Physics             |         1
```

FIGURE 25 – Nombre de cours dispensés dans le bâtiment du département

Question 21 : Titres des cours proposés et enseignants

La requête suivante permet d'afficher les titres des cours effectivement proposés dans une section ainsi que les enseignants qui les ont assurés.

```
SELECT DISTINCT c.title, t.name
FROM course c
JOIN section s
  ON c.course_id = s.course_id
JOIN teaches te
  ON s.course_id = te.course_id
 AND s.sec_id = te.sec_id
 AND s.semester = te.semester
 AND s.year = te.year
JOIN teacher t
  ON te.ID = t.ID;
```

```
SQL> SELECT DISTINCT c.title, t.name
  2  FROM course c
  3  JOIN section s
  4    ON c.course_id = s.course_id
  5  JOIN teaches te
  6    ON s.course_id = te.course_id
  7   AND s.sec_id = te.sec_id
  8   AND s.semester = te.semester
  9   AND s.year = te.year
 10  JOIN teacher t
 11    ON te.ID = t.ID;

TITLE                                              |NAME
---------------------------------------------------|------------
Intro. to Biology                                  |Crick
Genetics                                           |Crick
Intro. to Computer Science                         |Srinivasan
Intro. to Computer Science                         |Katz
Game Design                                        |Brandt
Robotics                                           |Srinivasan
Image Processing                                   |Katz
Image Processing                                   |Brandt
Database System Concepts                           |Srinivasan
Intro. to Digital Systems                          |Kim
Investment Banking                                 |Wu
World History                                      |El Said
Music Video Production                             |Mozart
Physical Principles                                |Einstein

14 lignes sÚlectionnÚes.
```

FIGURE 26 – Titres des cours proposés et enseignants correspondants

Question 22 : Nombre total de cours par période

La requête suivante calcule le nombre total de cours proposés pour chaque période Summer, Fall et Spring.

```
SELECT semester, COUNT(*) AS nb_cours
FROM section
WHERE semester IN ('Summer', 'Fall', 'Spring')
GROUP BY semester;
```

```
SQL> SELECT semester, COUNT(*) AS nb_cours
  2  FROM section
  3  WHERE semester IN ('Summer', 'Fall', 'Spring')
  4  GROUP BY semester;

SEMEST|  NB_COURS
------|----------
Summer|         2
Fall  |         3
Spring|        10
```

FIGURE 27 – Nombre total de cours par période

Question 23 : Crédits obtenus hors département

La requête suivante calcule, pour chaque étudiant, le nombre total de crédits obtenus dans des cours qui n'appartiennent pas à son propre département. Les cours non validés (note nulle ou égale à `F`) ne sont pas comptabilisés.

```
SELECT s.ID, s.name, NVL(SUM(
    CASE
        WHEN c.dept_name <> s.dept_name
         AND tk.grade IS NOT NULL
         AND tk.grade <> 'F'
        THEN c.credits
        ELSE 0
    END
),0) AS total_credits_hors_dept
FROM student s
LEFT JOIN takes tk
  ON s.ID = tk.ID
LEFT JOIN course c
  ON tk.course_id = c.course_id
GROUP BY s.ID, s.name
ORDER BY s.ID;
```

```
SQL> SELECT s.ID, s.name, NVL(SUM(c.credits), 0) AS total_credits_hors_dept
  2  FROM student s
  3  LEFT JOIN takes tk
  4    ON s.ID = tk.ID
  5  LEFT JOIN course c
  6    ON tk.course_id = c.course_id
  7   AND c.dept_name <> s.dept_name
  8  WHERE tk.grade IS NOT NULL
  9    AND tk.grade <> 'F'
 10  GROUP BY s.ID, s.name
 11  ORDER BY s.ID;

ID   |NAME                |TOTAL_CREDITS_HORS_DEPT
-----|--------------------|-----------------------
00128|Zhang               |                      0
12345|Shankar             |                      0
19991|Brandt              |                      0
23121|Chavez              |                      0
44553|Peltier             |                      0
45678|Levy                |                      7
54321|Williams            |                      0
55739|Sanchez             |                      0
76543|Brown               |                      0
76653|Aoi                 |                      0
98765|Bourikas            |                      7
98988|Tanaka              |                      0

12 lignes sÚlectionnÚes.
```

FIGURE 28 – Crédits obtenus par chaque étudiant hors de son département

Question 24 : Total des crédits des cours ayant eu lieu dans le département

La requête suivante calcule, pour chaque département, la somme des crédits des cours ayant effectivement eu lieu dans le bâtiment de ce département.

```
SELECT d.dept_name, NVL(SUM(c.credits), 0) AS total_credits
FROM department d
LEFT JOIN course c
  ON d.dept_name = c.dept_name
LEFT JOIN section s
  ON c.course_id = s.course_id
 AND s.building = d.building
GROUP BY d.dept_name
ORDER BY d.dept_name;
```

```
SQL> SELECT d.dept_name, NVL(SUM(c.credits), 0) AS total_credits
  2  FROM department d
  3  LEFT JOIN course c
  4    ON d.dept_name = c.dept_name
  5  LEFT JOIN section s
  6    ON c.course_id = s.course_id
  7   AND s.building = d.building
  8  GROUP BY d.dept_name
  9  ORDER BY d.dept_name;

DEPT_NAME           |TOTAL_CREDITS
--------------------|-------------
Biology             |           11
Comp. Sci.          |           21
Elec. Eng.          |            3
Finance             |            3
History             |            3
Music               |            3
Physics             |            4

7 lignes sÚlectionnÚes.
```

FIGURE 29 – Total des crédits des cours ayant eu lieu dans chaque département